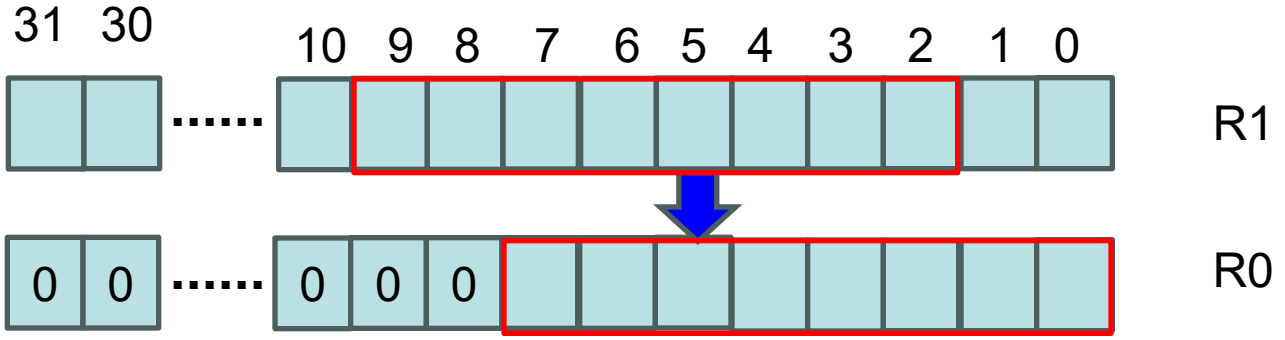
Homework #2 (1)

- Write ARM assembly programs that do the following computation and put the result at register r0. (不考慮 overflow / underflow)

Homework #2 (2)

- Q2-1: Please use "**shift、add、sub、or logic operations**" in the computation. (**DO NOT** use "multiply instruction")
  - r0 = r1 + 9*r2 – 16*r3
- Q2-2: `R0[7:0] ← R1[9:2]`
  `R0[31:8] ← 0`
  - Extract bits 2 through 9 from register R1 and store them into bits 0 through 7 of register R0.



Homework #2 (3)

- The values of r1, r2, and r3 are signed integers.
- The initial values of r1, r2, and r3 are assigned by yourself.

ARM CPU 使用「二補數 (Two's Complement)」來表示有號整數。在二補數系統下，無論你將數字視為有號還是無號，底層的二進位加法運算邏輯是完全相同的。ALU 只需要單純地執行 32-bit 的二進位加法即可。

Programmers 依照自己的需求，由 CPSR flag 來判斷:

- C (Carry) ： 判斷無號整數 (unsigned) 加法是否發生進位/溢位。
- V (Overflow) ： 判斷有號整數 (signed) 加法是否發生溢位。
- N (Negative) ： 判斷結果的最高位元（bit 31）是否為 1，通常用於表示有號數的負數。
- Z (Zero) ： 判斷結果是否為零（兩者皆適用）。

Template

```
/* ============================ */
/*         TEXT section         */
/* ============================ */
        .section .text
        .global main
        .type main,%function
main:
        mov   r1, #10          /* r1 = 10 */

        Your codes

        nop
        .end
```

- 一開始指定給r1, r2, r3的數值
- 助教批改作業時, 可能會測試不同的數值
- 因為編碼的緣故，不是每個數都能表示。請直接在GUI上修改register的值。

- #num : 表示10進位數字
- #0xnum : 表示16進位數字
- #0bnum: 表示2進位數字
- #0num: 表示8進位數字

Template (1)

```
/* ============================ */
/*        TEXT section          */
/* ============================ */
        .section .text
        .global main
        .type main,%function
main:
        mov   r1,#10          /* r1 = 10 */
        mov   r2,#20          /* r2 = 20 */
        mov   r3,#12          /* r3 = 12 */

        Your codes

        nop
        .end
```

程式請加上適當註解。

執行到nop時，r0的值為答案。

Pseudo Instruction: NOP

- This pseudo op will always evaluate to a legal ARM instruction that does nothing.
- Currently it will evaluate to **MOV r0, r0**.

Pseudo Instruction: LDR <register>, =<exp>

- **`LDR <register>,=label`**
  - If you plan to reference labels in other sections of code
- **`LDR <register>,=constant`**
  - If expression evaluates to a numeric constant then a MOV or MVN instruction will be used in place of the LDR instruction, if the constant can be generated by either of these instructions. Otherwise, the constant will be placed into the nearest literal pool (if it not already there) and a PC relative LDR instruction will be generated.

Template (2)

```
/* ============================ */
/*        TEXT section          */
/* ============================ */
        .section .text
        .global main
        .type main,%function
main:
        ldr   r1,=#10          /* r1 = 10 */
        ldr   r2,=#20          /* r2 = 20 */
        ldr   r3,=#12          /* r3 = 12 */

        Your codes

        nop
        .end
```

程式請加上適當註解。

執行到nop時，r0的值為答案。

Homework #2 (4)

- How to compile:

```
$ arm-none-eabi-gcc -g -O0 hw2-1.s -o \
hw2-1.exe
```

- How to execute

```
$ arm-none-eabi-insight
```

Homework #2 (5)

- Program should be assembled and linked by GNU cross toolchain.
- Program can be executed under **GDB ARM simulator**
- **程式中應有適當的說明（註解）**
- You should turn in to **ECOURSE2**
  - "**README.txt**" of "**README.pdf**": 描述你程式的內容、如何編譯程式、程式的執行環境、如何執行你的程式 (特別註明你的執行環境是否為Mac系統)
  - **hw2-1.s**、**hw2-2.s**: Your ARM assembly program
  - **hw2-1.exe**、**hw2-2.exe**: 編譯好的執行檔
  - 請將欲繳交的檔案壓縮成 <**hw2_學號.tar.xz**>，上傳壓縮檔

作業繳交截止日期為10月14日 (三) 24:00